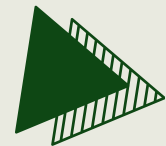


ADVANCED TASK MANAGER WEB APP TESTING

Software Testing CCSW 323



PREPARED BY :

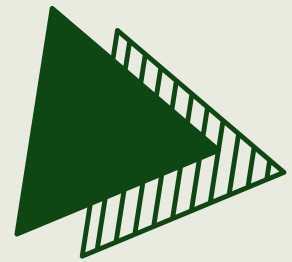
Abrar Alharbi-2317129

Ghala Alharbi-2310582

Elaf Alsharif-2310115

Toleen wael -2311776

Khadija janbi-2312308



introduction

- *Project: Software Testing of a Web Application*
- *Apply testing models and coverage criteria*
- *Design and execute test cases*
- *Use automation tools (Selenium & JUnit)*
- *Evaluate system functionality and reliability*

SOFTWARE UNDER TEST

The To-Do List Web Application is a simple web-based system that helps users manage and organize their daily tasks efficiently.

- **Built using:** HTML, CSS, JavaScript
- **System Size:** Approximately 320 Lines of Code (LOC)
- **Goal:** To ensure that the system meets the required specifications, supports user needs, and prevents invalid or incorrect user inputs to maintain accurate operation.

SOFTWARE UNDER TEST

Application Features

- User login system
- Add new tasks
- Edit existing tasks
- Mark tasks as completed
- Delete tasks
- Search and filter tasks

SOFTWARE UNDER TEST

Main page after successful login.

 **My Tasks**





Add Task

Logout

AllCompletedPending

SOFTWARE UNDER TEST

After adding three different priority tasks.

 **My Tasks**

☐ test 1 (Due: 2026-04-24)

Edit X

☐ test 2 (Due: 2026-04-25)

Edit X

☐ test 3 (Due: 2026-04-25)

Edit X

☐ test 4 (Due: 2026-04-25)

Edit X

TESTING MODELS

1. Graph-Based Testing

- Test user actions (flow)
- Example: add → complete → delete

2. Input Space Testing

- Test different inputs
- Example: empty, wrong, valid

COVERAGE CRITERIA

GRAPH-BASED TESTING

NC (Node Coverage)

- Test all system states
- Add / Edit / Delete / Complete

EC (Edge Coverage)

- Cover each transition
- Test all system flows

COVERAGE CRITERIA

INPUT SPACE

BCC (Base Choice Coverage)

- Start with normal case
- Change one input each time

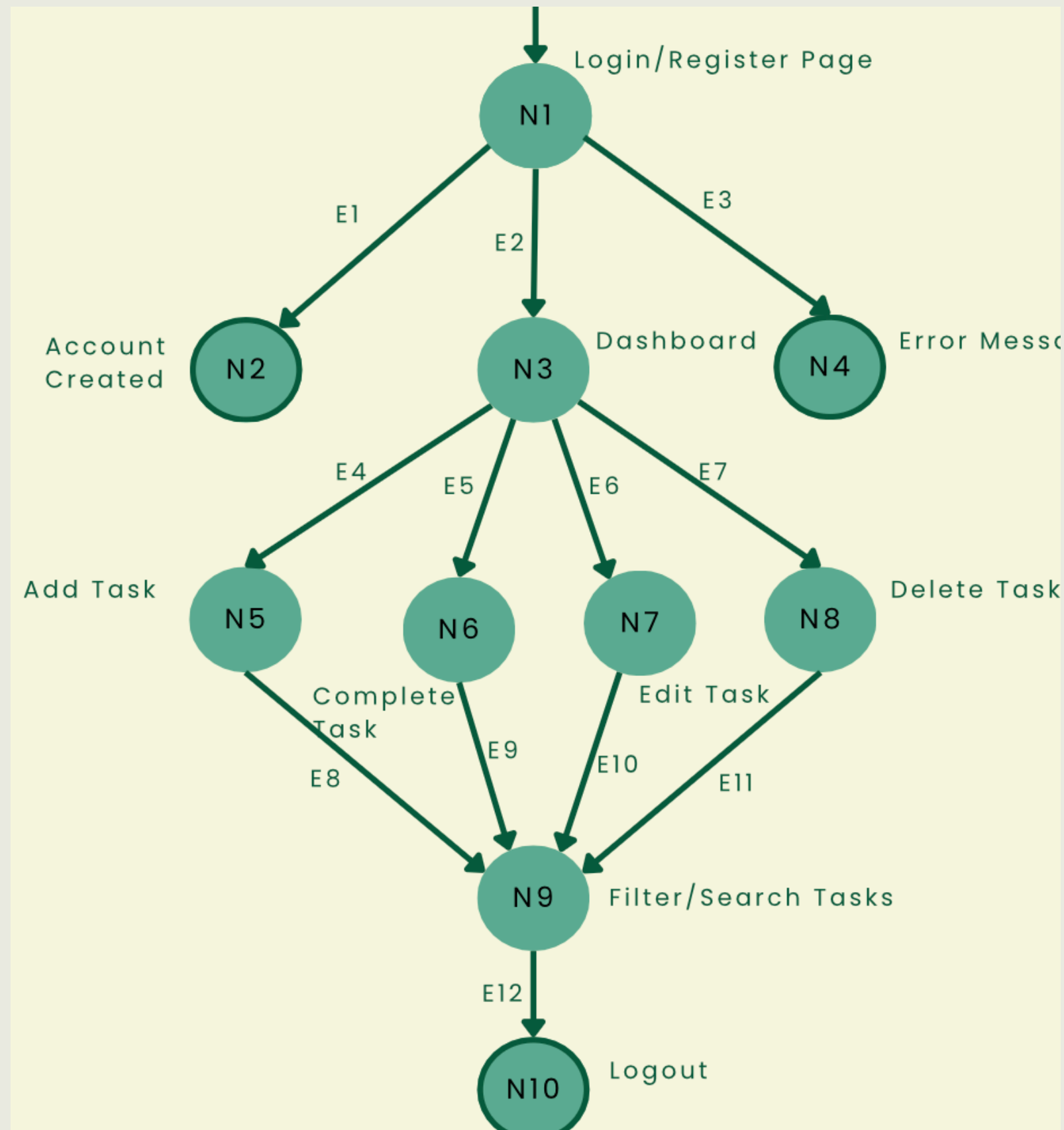
ECC (Each Choice Coverage)

- Test each input value once
- Cover all input types (valid, invalid, empty)

multiple base choice Coverage (MBCC)

- Use more than one normal case as a base
- Vary inputs around each base case to increase coverage

SYSTEM GRAPH MODEL



COVERAGE CRITERIA

Node Coverage (NC)

Test Case ID	Input Data	Node Covered	Expected Output	Actual Output
NC-01	Enter a new valid email and password, then click Create Account	N1, N2	Account is created successfully	Account created successfully
NC-02	Enter valid email and password, then click Login	N1, N3	User is redirected to dashboard	Login successful

COVERAGE CRITERIA

Edge Coverage (EC)

Test Case ID	Input Data	Edge Covered	Expected Output	Actual Output
EC-01	Enter a new valid email and password, then click Create Account	E1	Account created message shown	Account created successfully
EC-02	Enter valid email and password, then click Login	E2	User is redirected to dashboard	Login successful

COVERAGE CRITERIA

Base Choice Coverage (BCC)

**Input: Task Manager
Login**

Task Manager Login

Login

Create Account

COVERAGE CRITERIA

Base Choice Coverage (BCC)

Characteristics: Email, Password	BCC Test Cases: base = {a1 ,b1} A {a2 ,b1} A {a3 ,b1} B {a1 ,b2} B {a1 ,b3}
Blocks: email: (valid , empty,wrong) password: (valid, empty , wrong)	
Abstract IDM: A = [a1, a2, a3] , B = [b1, b2,b3]	
Number of tests: 1(base)+2+2 = 5	

COVERAGE CRITERIA

Base Choice Coverage (BCC)

Test Case ID	Input Data	Expected Output	Actual Output
BCC-01	Email: aa123@gmail.com, Password: 123456	User is redirected to the dashboard and login is successful	login is successful
BCC-02	Email: (empty), Password: 123456	System shows error message and login is not allowed	"Invalid credentials" message displayed

COVERAGE CRITERIA

Each Choice Coverage(ECC)

Input: add tasks

 **My Tasks**

Task name

dd/mm/yyyy

High Priority

Add Task

Logout

COVERAGE CRITERIA

Each Choice Coverage(ECC)

Characteristics: name task , due date ,priority	ECC Test Cases {a1 , b1, c1} {a2, b2, c2} {a1, b1, c3} NOTE :3 test cases were sufficient to cover all input characteristics and their corresponding blocks
Blocks: name taske : (valid , empty) due date : (valid, empty) priority:(high,medium ,low)	
Abstract IDM: A = [a1, a2] , B = [b1, b2] , C = [c1,c2,c3]	
• Number of tests: MAX(2,2,3) =3	

COVERAGE CRITERIA

Each Choice Coverage(ECC)

Test Case ID	Input Data	Expected Output	Actual Output
ECC-01	Task: "study ch1", Date: 20/04/2026, Priority: High	Task is added successfully	Task added successfully
ECC-02	Task: "", Date: (empty)Priority: medium	Task NOT added, validation message shown	Validation message displayed

COVERAGE CRITERIA

multiple base choiceCoverage(MBCC)

Input: add tasks

 My Tasks

Task name

dd/mm/yyyy

High Priority

Add Task

Logout

COVERAGE CRITERIA

multiple base choiceCoverage(MBCC)

Characteristics: name task , due date ,priority	MBCC Test Cases BASE1 {a1 , b1, c1} BASE2 {a2, b2, c2} Tests from a1, b1,c1: a1, b1, c3, Tests from a2,b2,c2: a2, b2, c3,
Blocks: name taske : (valid , empty) due date : (valid, empty) priority:(high,medium ,low)	
Abstract IDM: A = [a1, a2] , B = [b1, b2] , C = [c1,c2,c3]	

COVERAGE CRITERIA

multiple base choice Coverage (MBCC)
base 1 & 2

Test Case ID	Input Data	Expected Output	Actual Output
MBCC-01	Task: "study chl", Date: 20/04/2026, Priority: High	Task added successfully with high priority	Task added successfully
MBCC-02	Task: "", Date: (empty)Priority: medium	Task NOT added, validation message shown	Validation message displayed

COVERAGE CRITERIA

multiple base choice Coverage (MBCC)
test from base 1 & 2

MBCC-03	Task: "Gym", Date: 22/04/2026, Priority: Low	Task added successfully with low priority	Task added successfully
MBCC-04	Task: "", Date: (empty)Priority: Low	Task NOT added, validation message shown	Validation message displayed

TESTING TOOL



They automate test execution, verify system behavior accurately, and improve testing efficiency and reliability.

TESTING TOOL

Selenium WebDriver

Used to automate browser actions and simulate user behavior.



JUnit

Used to execute test cases and verify expected results automatically.



TOOL USAGE

Selenium WebDriver Features

- Automates browser actions
- Supports multiple browsers
- Executes test cases automatically
- Simulates real user behavior



TOOL USAGE

Usage in Our Project

- Opened the web application
- Performed login operations
- Added tasks automatically
- Verified system responses



TOOL USAGE

JUnit Features

- Runs automated test cases
- Checks expected vs actual results
- Displays pass/fail status
- Generates test reports

The JUnit logo is displayed in a bold, green, sans-serif font. The letters 'J' and 'U' are significantly larger than the letters 'n' and 'i'. The logo is centered within a white rectangular box.

TOOL USAGE

Usage in Our Project

- Executed automated test cases
- Verified system behavior
- Displayed test results
- Confirmed successful execution

The JUnit logo, featuring the word "JUnit" in a bold, green, sans-serif font, centered within a white square.

TOOL USAGE

Installation

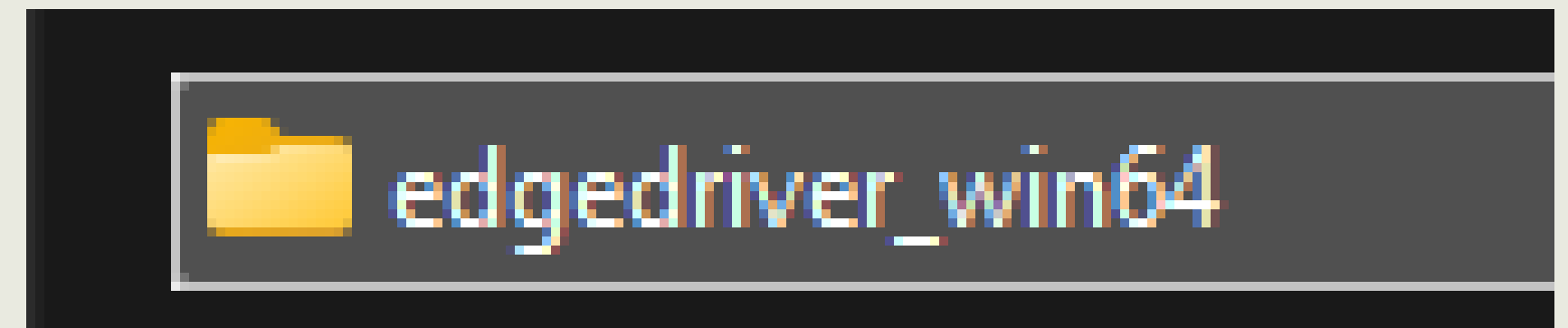
1. Download browser driver (e.g., EdgeDriver)
2. Add Selenium library to the project
3. Configure the driver path in the code
4. Add JUnit library to the project
5. Create a test class
6. Use @Test annotation to define test cases



TOOL USAGE

Installation

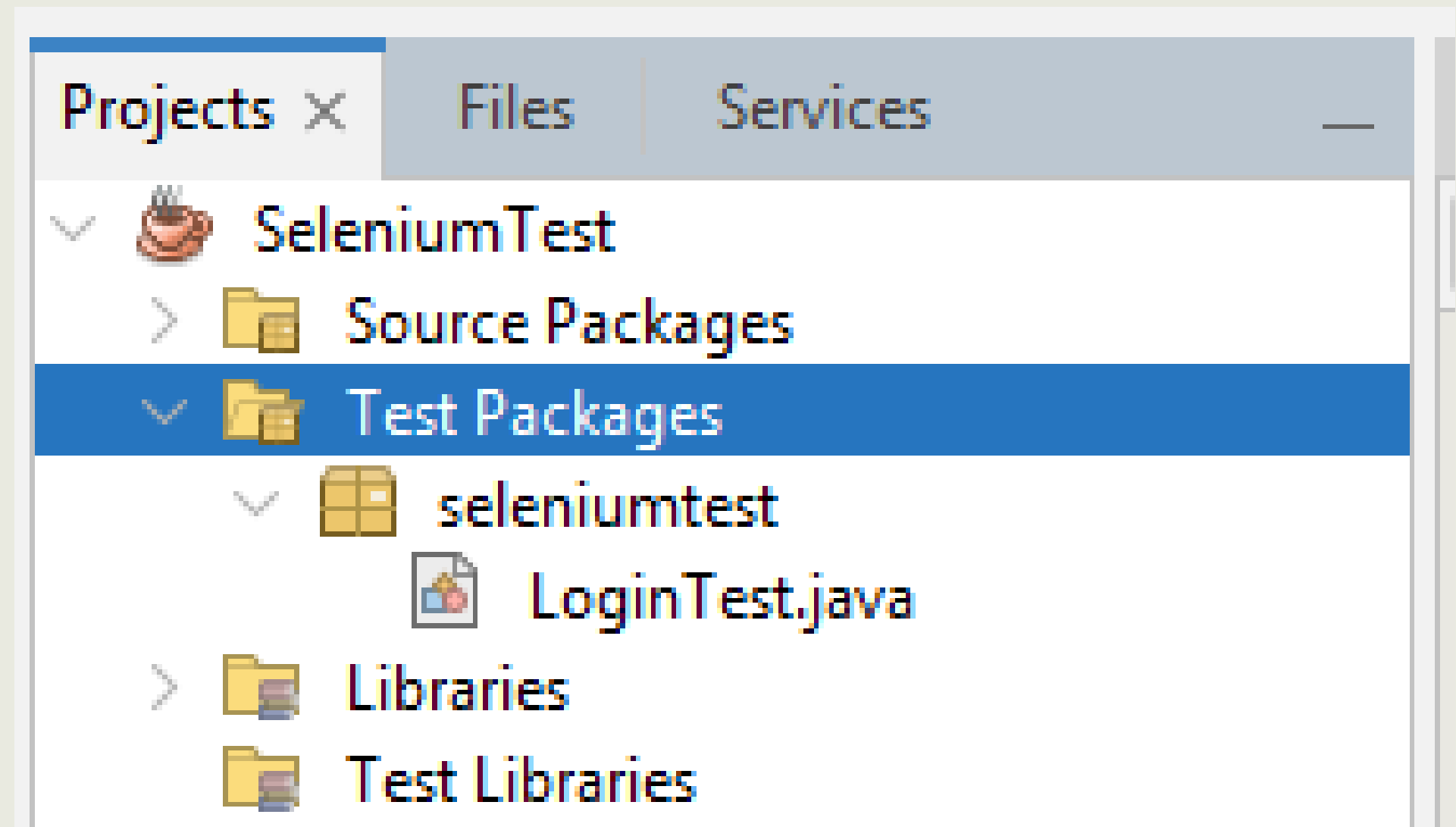
1. Download browser driver (e.g., EdgeDriver)



TOOL USAGE

Installation

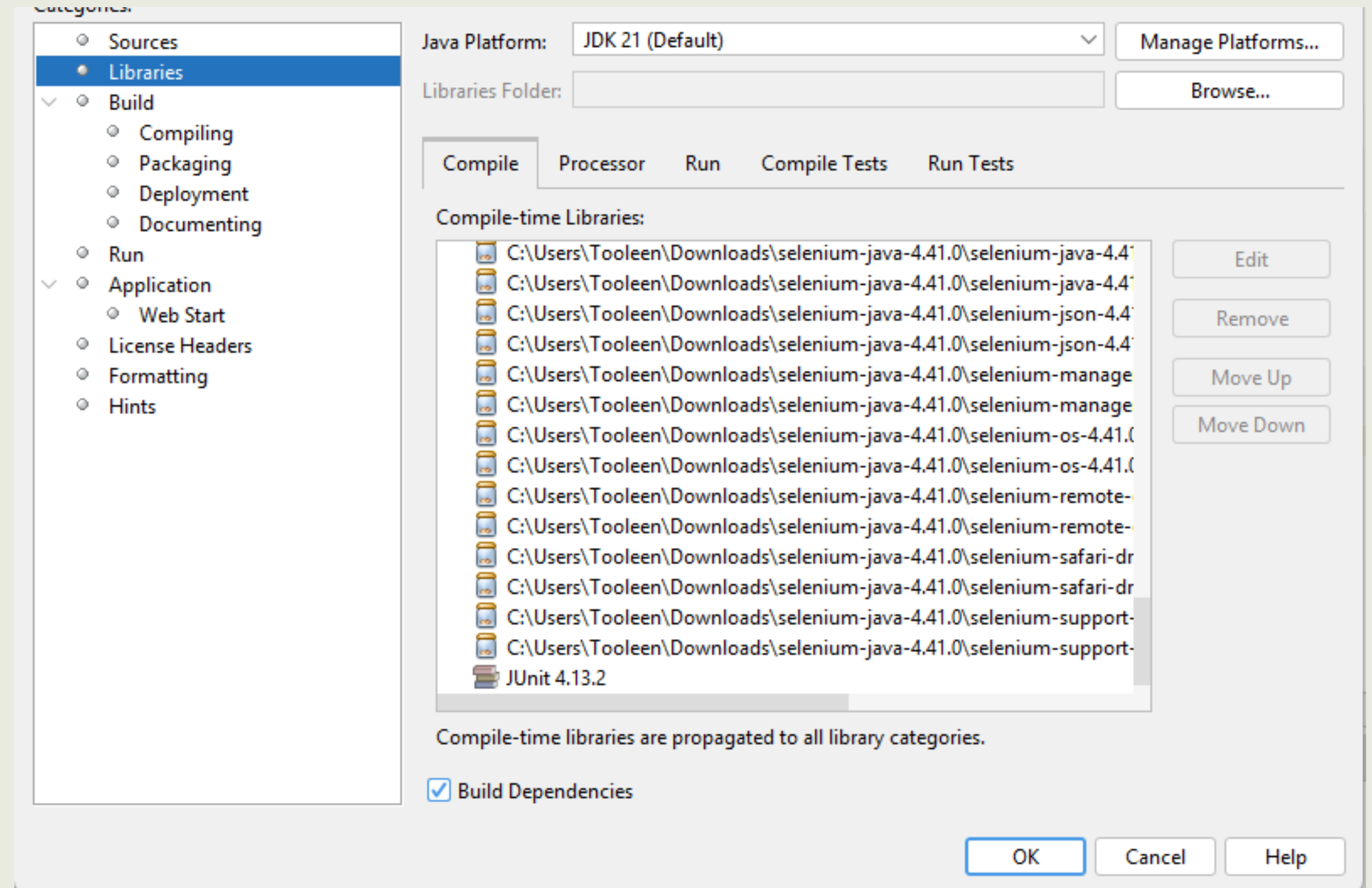
2. Create a test class



TOOL USAGE

Installation

3. Add JUnit and Selenium libraries to the project



TOOL USAGE

Installation

4.contains test methods using Selenium EdgeDriver and JUnit annotations to automate test execution.

```
4
5 public class LoginTest {
6
7     @Test
8     public void testCreateAccount_LoginValidation_AddTask() throws InterruptedException {...98 lines }
9
10 }
```


TOOL USAGE

Example Test Case Execution Using Selenium and JUnit

TOOL USAGE

Test Case ID: BCC-02

Description:

This test case verifies the system behavior when the user attempts to login with an empty email field.

TOOL USAGE

Test Case ID: BCC-02

Description:

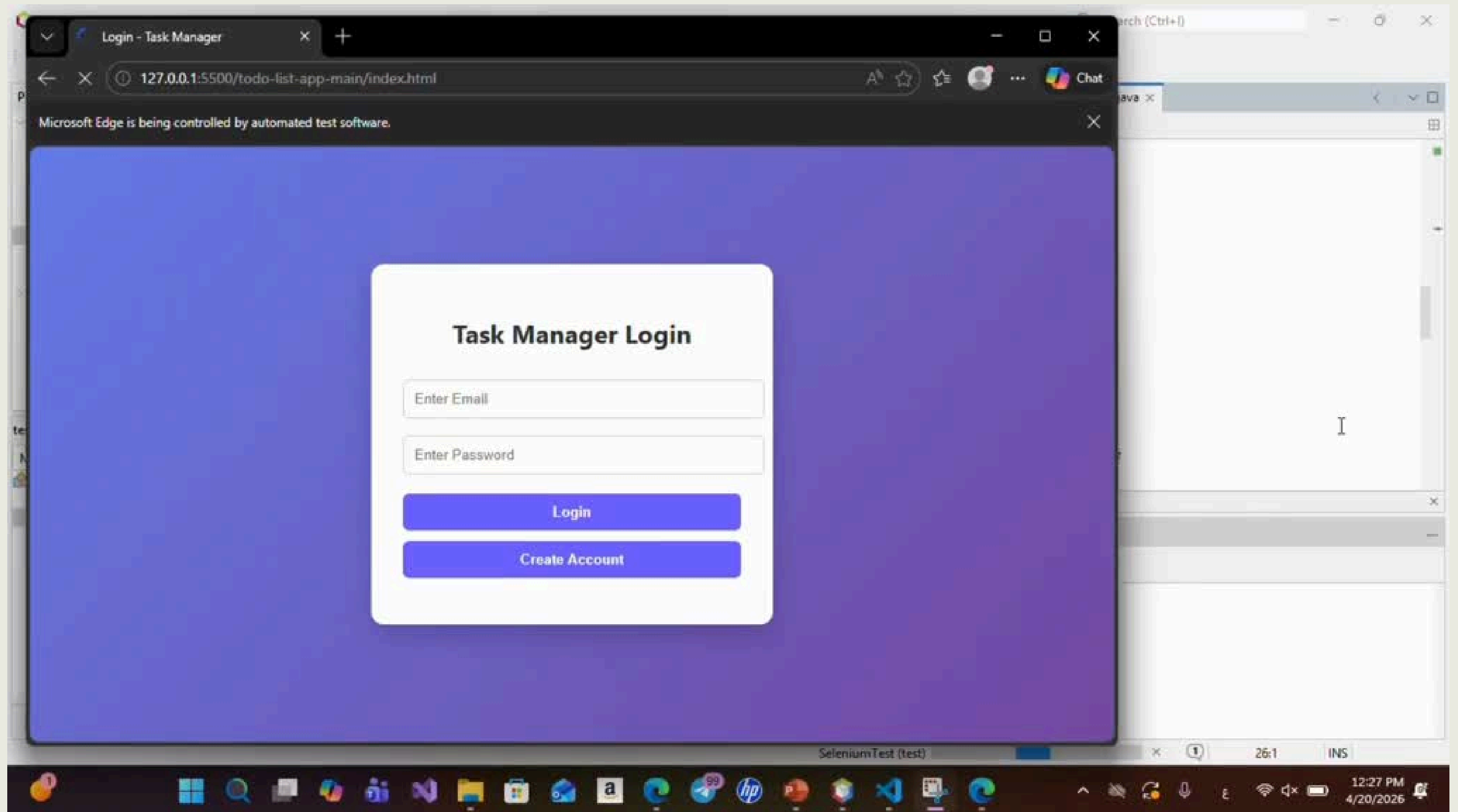
This test case verifies the system behavior when the user attempts to login with an **empty** email field.

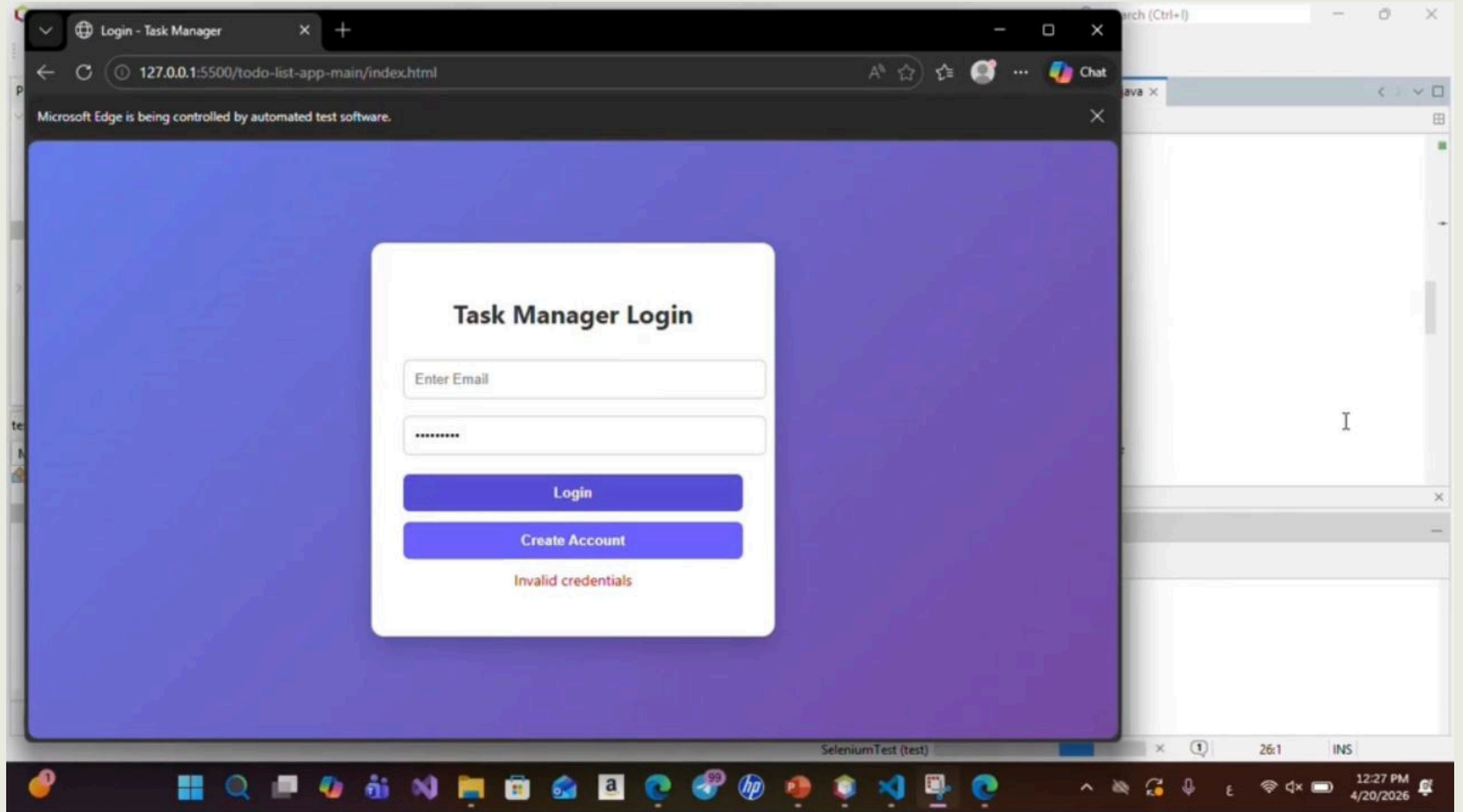
TOOL USAGE

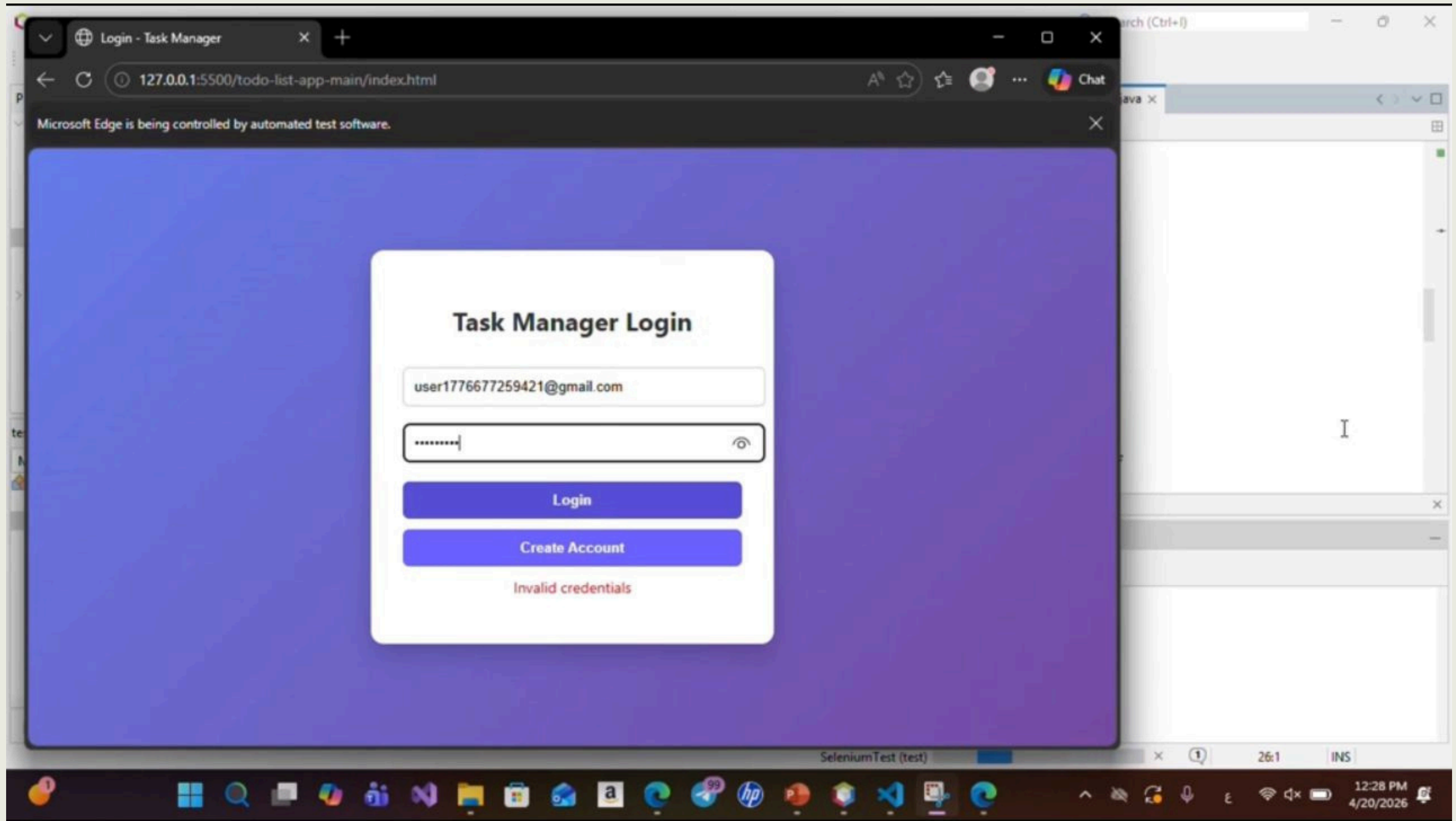
Test Case ID: ECC-01

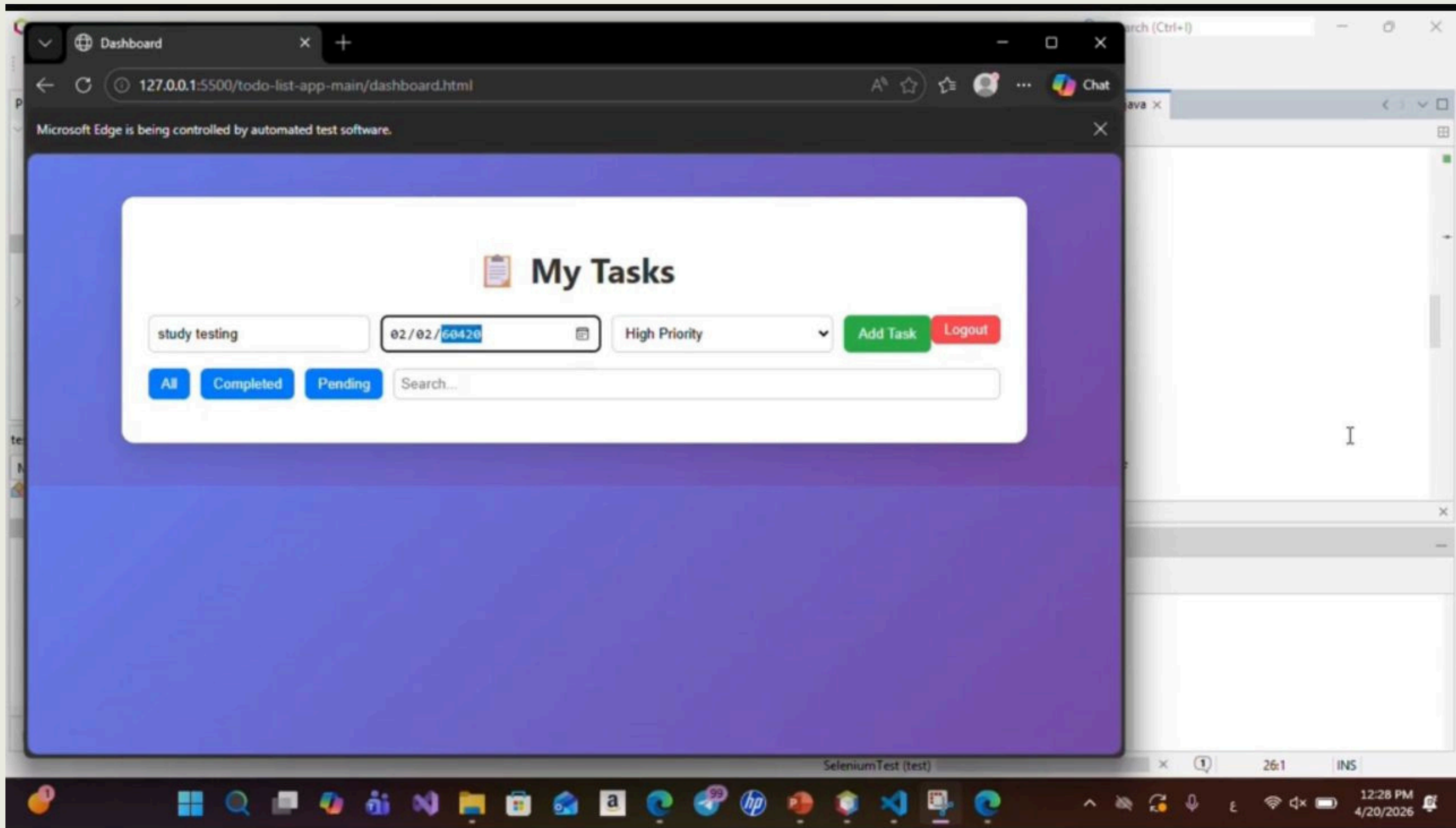
Description:

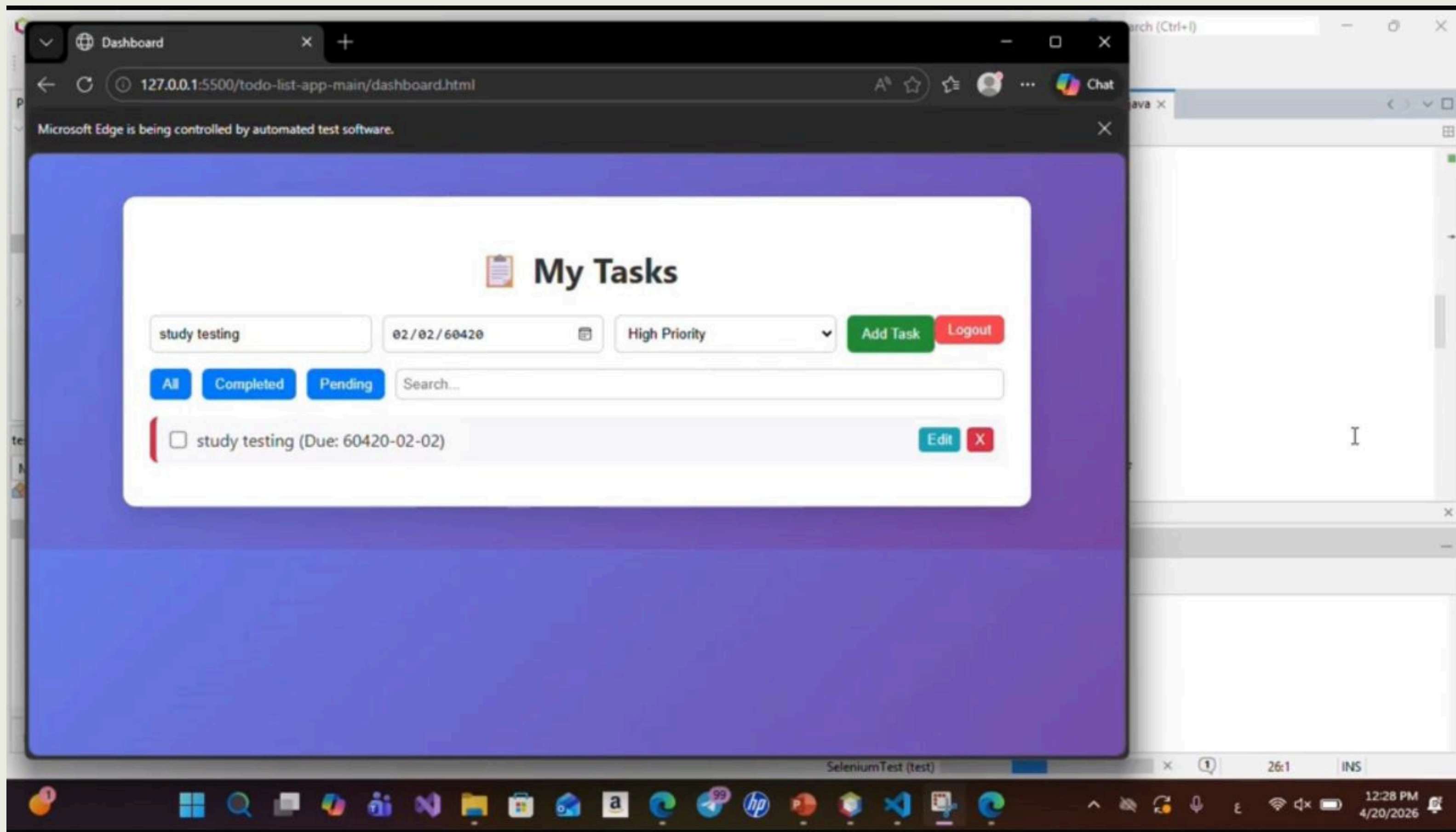
This test case verifies that the system allows the user to **add** a new task successfully using valid input data.











FileEditViewNavigateSourceRefactorRunDebugProfileTeamToolsWindowHelp

SeleniumTest - Apache NetBeans IDE 25

Search (Ctrl+I)

ProjectsFilesServices

SeleniumTest

- Source Packages
 - seleniumtest
 - SeleniumTest.java
- Test Packages
 - seleniumtest
 - LoginTest.java
- Libraries
- Test Libraries
- TestingLab5

testCreateAccount_LoginValidation_Ad...

Members<empty>

LoginTest

- LoginTest()
- testCreateAccount_LoginValidation_AddTa

SourceHistory

```
52
53 // -----
54 // Step 2: Example 1
55 // Login without email
56 // -----
57 emailField = driver.findElement(By.id("email"));
58 passwordField = driver.findElement(By.id("password"));
59
60 emailField.clear();
61 passwordField.clear();
62
63 passwordField.sendKeys(testPassword);
64
65 driver.findElement(By.cssSelector("button[onclick='login()']")).click();
66
67 authMessage = wait.until(
68     ExpectedConditions.visibilityOfElementLocated(By.id("authMessage")));
69
```

Test ResultsOutput - SeleniumTest (test)

seleniumtest.LoginTest

- Tests passed: 100.00 %
- The test passed. (74.042 s)

New account created successfully
Example 1 Passed
Login successful
Example 2 Passed - Task added succ
Apr 20, 2026 12:27:39 PM org.openq
WARNING: Unable to find an exact n

Finished building SeleniumTest (test).

26:1INS12:28 PM 4/20/2026

TEST RESULTS

PASS/FAIL RESULTS:

ALL 28 TEST CASES PASSED SUCCESSFULLY, INDICATING THAT THE SYSTEM OPERATES CORRECTLY ACCORDING TO THE DEFINED REQUIREMENTS.

NUMBER OF PASSED TEST CASES: 28 (100%)

NUMBER OF FAILED TEST CASES: 0 (0%)

All functions worked correctly

CONCLUSION

- Selenium was effective for automated testing
- Helped save time and improve accuracy
- Main features were tested successfully
- More test cases can be added in the future



Thank you.